UNIVERSITÉ LIBRE DE BRUXELLES
Faculté des Sciences — Département d'Informatique

Détection de communautés dans DBLP

INFO-F-203 — Algorithmique 2
Projet Académique 2025–2026

Étudiants :
[GHERASIM Sorin]
Matricule : [000594364]

[MURYANGO Blaise]
Matricule : [000599305]

Cours :
INFO-F-203

Année académique 2025–2026

Table des matières

1 Introduction **2**

2 Méthodologie **2**
2.1 Architecture du code 2
2.2 Tâche 1 — Communautés de co-publication 2
2.3 Tâche 2 — Communautés orientées 2

3 Résultats **2**
3.1 Tâche 1 3
3.2 Tâche 2 3
3.3 Tâche Bonus 3

4 Discussion **4**

5 Conclusion **4**

1 Introduction

Ce rapport présente le travail réalisé dans le cadre du projet du cours INFO-F-203 Algorithmique 2. L'objectif est d'analyser la base de données DBLP (snapshot du 1er janvier 2026, environ 8 millions de publications) afin d'identifier des communautés de chercheurs selon deux approches :

1. des communautés de co-publication via un graphe non orienté maintenu en ligne avec *Union-Find*.
2. des communautés orientées basées sur les composantes fortement connexes (CFC) d'un graphe filtré (seuil ≥ 6 co-publications), identifiées par l'algorithme de Tarjan.

Une tâche bonus détermine les pays d'affiliation des auteurs via l'API OpenAlex.

2 Méthodologie

2.1 Architecture du code

Le projet est structuré en packages Java : `dblp.core` contient la classe abstraite `Tache` qui factorise le parsing en flux et le mapping auteur/id. `dblp.structures` contient les structures de données (*Union-Find*, graphe orienté, Tarjan) ; `dblp.taches` contient les implémentations concrètes. Ce patron *Template Method* permet aux deux tâches de partager le même code de parsing tout en définissant chacune sa propre logique de traitement.

2.2 Tâche 1 — Communautés de co-publication

Algorithme : *Union-Find* avec compression de chemin et union pondérée par taille. Pour chaque publication, tous les co-auteurs sont fusionnés dans la même composante. L'état est cohérent après chaque publication (traitement online).

Structures de données : Le *Union-Find* utilise deux tableaux (`parent[]` et `size[]`) pré-alloués à 5 millions d'entrées. Un `HashMap<String, Integer>` associe chaque nom d'auteur à un identifiant numérique.

Complexité : Les opérations *find* et *union* s'exécutent en $O(\alpha(n))$ amorti grâce à la compression de chemin, où α est la fonction inverse d'Ackermann. Pour m publications avec k auteurs chacune, la complexité totale est $O(m \cdot k \cdot \alpha(n))$, quasi-linéaire. La mémoire est $O(n)$ avec $n \approx 3,9$ millions d'auteurs.

2.3 Tâche 2 — Communautés orientées

Comptage en ligne : Durant le parsing, pour chaque publication avec ≥ 2 auteurs, on incrémente un compteur pour chaque paire ordonnée (premier auteur $\rightarrow$ co-auteur). Les deux identifiants sont encodés dans un seul `long` via décalage de bits, servant de clé dans un `HashMap`.

Graphe filtré : Après le parsing, on ne conserve que les arêtes $A \rightarrow B$ dont le compteur est ≥ 6. Le graphe est représenté par liste d'adjacence.

CFC : L'algorithme de Tarjan (version itérative pour éviter les dépassements de pile) identifie les composantes fortement connexes en $O(V + E)$.

Diamètre : Pour chacune des 10 plus grandes CFC, un BFS depuis chaque sommet du sous-graphe induit donne la distance maximale. Pour les CFC > 500 sommets, on échantillonne 500 sources. Complexité : $O(|C| \cdot (|C| + |E_C|))$ par communauté.

3 Résultats

3.1 Tâche 1

Le programme a traité 7 995 940 publications en environ 50 secondes, identifiant environ 3,9 millions d'auteurs distincts. L'histogramme des tailles (voir Figure 1) montre une distribution fortement asymétrique : la grande majorité des communautés sont de petite taille, tandis qu'une composante géante regroupe la majorité des auteurs. Ce résultat est conforme aux propriétés connues des réseaux de co-publication.

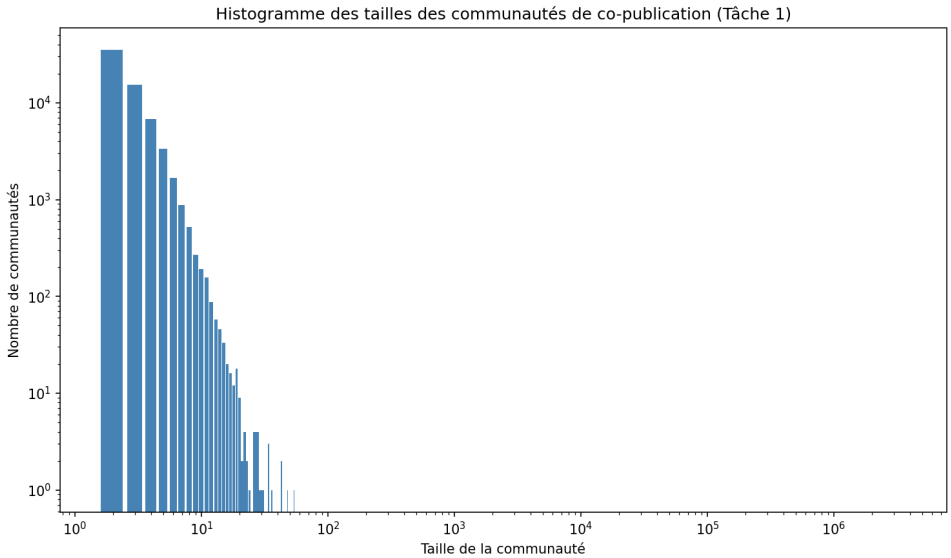

FIGURE 1 – Histogramme des tailles des communautés de co-publication (Tâche 1).

3.2 Tâche 2

Après filtrage (seuil ≥ 6), le graphe orienté contient 272 861 sommets. L'algorithme de Tarjan identifie 260 206 CFC. Le Tableau 1 présente les 10 plus grandes.

TABLE 1 – Les 10 plus grandes CFC du graphe orienté filtré (seuil ≥ 6), avec leur taille et leur diamètre.

Rang	Taille	Diamètre
1	133	30
2	73	35
3	38	13
4	34	7
5	33	6
6	30	17
7	27	10
8	26	12
9	23	6
10	21	7

3.3 Tâche Bonus

L'API OpenAlex a été interrogée pour chaque auteur des 10 plus grandes CFC afin d'identifier leur pays d'affiliation. Le graphique (voir Figure 2) montre que certaines communautés sont

fortement associées à un pays (par exemple, la CFC 10 est majoritairement italienne), tandis qu'autres sont plus internationales.

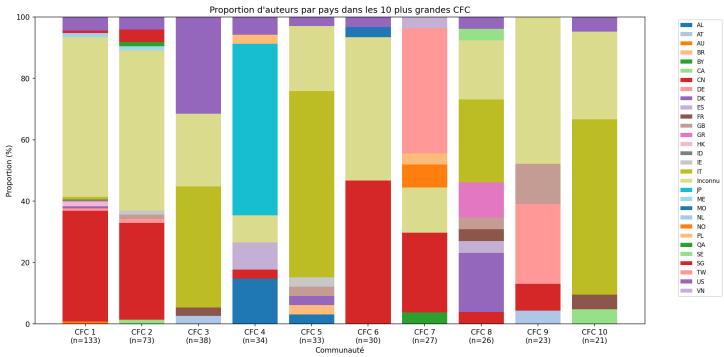

FIGURE 2 – Proportion d'auteurs par pays dans les 10 plus grandes CFC.

4 Discussion

Limites : Le calcul exact du diamètre est quadratique en la taille de la CFC. Pour les CFC > 500 sommets, l'échantillonnage donne une borne inférieure. Le *Union-Find* pré-alloue 5 millions de slots, ce qui consomme de la mémoire même si tous ne sont pas utilisés. La résolution des noms via OpenAlex est imparfaite (homonymes, auteurs non indexés).

Conformité des résultats : La composante géante dans la Tâche 1 est attendue pour un réseau de co-publication (propriétés de réseau petit-monde). Pour la Tâche 2, le seuil de 6 filtre efficacement les relations faibles et fait émerger des groupes de recherche stables. Les diamètres élevés (jusqu'à 35) indiquent des chaînes de collaboration longues mais cohérentes.

Optimisations pour OpenAlex (480M publications, 120M auteurs) :

— Remplacer le `HashMap` en mémoire par une structure sur disque (RocksDB).
— Utiliser un hachage parfait minimal pour les identifiants.
— Paralléliser le calcul des diamètres.
— Utiliser l'algorithme de double-BFS (2-sweep) pour approximer le diamètre en temps linéaire.

5 Conclusion

Ce projet a permis de mettre en pratique les structures *Union-Find* et l'algorithme de Tarjan sur un jeu de données réel de grande taille. Les deux approches révèlent des structures communautaires à différentes échelles. La tâche bonus montre que certaines communautés sont fortement localisées géographiquement. Les pistes d'amélioration incluent le stockage sur disque et l'approximation algorithmique du diamètre pour le passage à l'échelle.

Références

[1] dblp TEAM. *dblp computer science bibliography*. 2026. URL : https://dblp.org/.

[2] dblp TEAM. *Monthly Snapshot XML Release of January 2026*. Jan. 2026.

[3] dblp TEAM. *XML Document Type Definition (DTD) Release of May 2023*. Mai 2023.

[4] M. LEY. "Dblp : some lessons learned". In : *Proc. VLDB Endow.* 2.2 (2009), p. 1493-1500.

[5] J. PRIEM, H. PIWOWAR et R. ORR. *OpenAlex : A fully-open index of scholarly works*. 2022. URL : https://openalex.org/.